

“Figma is a big black box:” The mismatch between the interface and its underlying structure

Carl Abou Saada Nujaim

ExSitu, LISN

Université Paris-Saclay, CNRS, Inria

Orsay, France

carl@lisn.fr

Michel Beaudouin-Lafon

ExSitu, LISN

Université Paris-Saclay, CNRS, Inria

Orsay, France

mbl@lisn.fr

Victoria MeiFeng Myot

ExSitu, LISN

Université Paris-Saclay, CNRS, Inria

Orsay, France

victoria.myot@universite-paris-saclay.fr

Wendy E. Mackay

ExSitu, LISN

Université Paris-Saclay, CNRS, Inria

Orsay, France

wendy.mackay@inria.fr

Abstract

High-fidelity digital prototyping tools play a central role in commercial design workflows. One of the most common tools is FIGMA, which incorporates programming-like concepts such as constraints and parameterization in the form of *auto layout*, *components*, and *smart animate* features. We studied how designers engage with and appropriate these concepts as they used FIGMA to prototype an interactive system. We found recurring breakdowns that stem from a mismatch between the *interface*, which encourages imperative manipulation of objects on a canvas, and the *underlying declarative structure*, which is governed by constraints. Based on these findings, we characterize FIGMA as a complex *interaction substrate* and derive implications for the design of future constraint-reliant tools.

CCS Concepts

• **Human-centered computing** → *Empirical studies in HCI*.

Keywords

high-fidelity prototyping, creativity-support tools, generative theories of interaction, visual programming, constraints, designer workflows

ACM Reference Format:

Carl Abou Saada Nujaim, Victoria MeiFeng Myot, Michel Beaudouin-Lafon, and Wendy E. Mackay. 2026. “Figma is a big black box:” The mismatch between the interface and its underlying structure. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798460>

1 Introduction

Digital high-fidelity prototyping tools such as FIGMA¹ and ADOBE XD² play a central role in commercial design workflows [6, 9, 20] by supporting the rapid design of artifacts ranging from responsive websites to interactive applications. FIGMA accomplishes this through advanced features based on concepts traditionally associated with programming environments such as modularity, constraints, and parameterization. We focused on three illustrative examples of these features:

- *Auto layout*: structures *frames*—containers that hold objects—by arranging their content on a grid or in a horizontal or vertical layout, with rules to control spacing and margins;
- *Components*: act as reusable elements for generating linked *instances* that automatically update when their parent components change. Users can customize properties by editing text or boolean variables, and switch between *variants*; and
- *Smart animate*: creates animated transitions between frames by matching layers based on hierarchical ordering and naming.

As FIGMA is not explicitly presented as a visual programming system, this raises a fundamental question: *to what extent are designers able to adopt these visual programming concepts to prototype interactive systems?* We investigate how designers use FIGMA’s advanced features when prototyping an interactive application, and present implications for the design of future prototyping tools.

2 Related work

2.1 Designer practices in the industry

Previous studies of user experience (UX) designers provide a global view of human-centered design practices [10] and strategies for including users in the design process [7]. More specifically, researchers focused on collaboration practices and the challenges designers face, including when analyzing data [11] and when communicating with stakeholders, given that “business objectives [are prioritized over] fostering ethical outcomes” [18]. Maudet et al. [14]



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI EA '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/26/04

<https://doi.org/10.1145/3772363.3798460>

¹<https://figma.com>

²<https://adobexdplatform.com>

identify communication breakdowns between designers and developers, as they misinterpret each other's representations and intentions. Tools such as RELAY [12] attempt to bridge this gap by allowing designers to communicate their intent through annotations on their designs. This mismatch persists despite the rise of designer-targeted prototyping tools such as FIGMA [9, 20]. Chen and Yoon [6] explore designers' use of such platforms, highlighting the "role of all-in-one digital tools in centralizing designers' workflows" and the increasing focus on high-fidelity prototypes. Most of this research relies on interviews and surveys, and we lack direct studies of how professional designers interact with high-fidelity prototyping tools.

2.2 Constraints in visual programming

Visual programming tools allow users to specify a program by interacting with a visual representation in space [16] where editing visual elements can directly trigger computation [19]. Chatty et al. [5] introduced such a system to designers, enabling them to graphically program interfaces. Constraint programming is a declarative programming paradigm in which users specify relationships between entities instead of providing imperative instructions [17]. In visual programming systems, constraints frequently act as the computational glue between graphical elements: users manipulate visual objects that represent variables, while constraint relationships between them are maintained automatically by the system [15]. THINGLAB [3] illustrates this pattern through graphical objects that are connected by continuously-maintained constraints as users manipulate the scene. PERIDOT [15] allows designers to create interactive interfaces by directly manipulating graphics whose spatial and behavioral relationships are governed by constraints. More recently, STICKYLINES [8] enables designers to maintain alignment and spacing relationships through interactive guides, acting as persistent constraints. Despite these conceptual connections with tools such as FIGMA, we lack empirical evidence about how designers work with constraints in high-fidelity prototyping tools.

3 Method

We conducted a study of novice and expert designers as they created an interactive application in FIGMA. We recruited 13 participants (8 women, 5 men; aged 20–45) including both full-time UX designers and graduate students in HCI, computer science, or design. All participants had some prior experience with FIGMA. We classified nine of them as *novices* (*P*) as they used FIGMA occasionally, and four as *experts* (*E*) as they used the system daily. We followed our institution's IRB policy to conduct the study.

3.1 Procedure

The study takes approximately 1 hour. Participants complete a 30-minute task on their personal laptop using FIGMA, either remotely in their own environment or in-person in a large room with two or three other participants, where they were seated apart and did not interact during the task.

We first present a live, 10-minute tutorial on how to use FIGMA's auto layout, component, and smart animate features and answer any questions. We next explain the design task, which consists of prototyping a note application in FIGMA. Participants must develop

a *home screen* where users can see their recently-edited notes in a list, grid, or other structure, and a *note component* that includes different properties and different types of content, such as text, a checklist, or a picture. The home screen should connect to a *secondary screen* using the prototype feature. Participants should also use *auto layout* to make certain layouts responsive, and include a *continuous interaction*, such as a drag or a long press. Expert users are specifically asked to include a drag-and-drop interaction.

Participants have access to a base project containing ready-made elements and can ask questions or search the Internet if needed. We periodically check whether participants are blocked and provide minimal assistance to unblock progress, logging every intervention. We provide a "cheat sheet" containing the task description and information from the tutorial as well as a blank sheet of paper for them to record their thoughts as they perform the task. After completing the task, participants share their prototypes with each other, explaining their process and breakdowns.

3.2 Data collection and analysis

We collected 30-minute screen recordings of participants as they completed the task, as well as over-the-shoulder video recordings and their written comments. We analyzed the data using a reflexive thematic analysis [4] approach. Two authors first coded each entry and identified its context, including the feature—component, auto layout, or prototype—and sub-feature involved; the type of problem encountered; its perceived cause; the strategy the participant adopted to address it; and if external help was needed to solve it. We then transcribed all entries onto individual notes and conducted several rounds of manual grouping, identifying an initial set of recurring problem clusters. To gather deeper insights, we analyzed observations that went beyond surface-level issues, examining what participants attempted to do, how they acted in response to the system, and how the system interpreted their actions. Underlying patterns surfaced, leading us to synthesize a set of more focused themes.

4 Results: FIGMA as a declarative structure

We identified a recurring mismatch between how FIGMA's interface presents interaction to users and how the system is structured underneath, which E13 described as a "big black box." At the surface level, FIGMA emulates common layouts in popular design tools, such as ADOBE PHOTOSHOP³ and ILLUSTRATOR⁴. Figure 1 presents a large canvas with a toolbar (B), a side panel to configure elements (C), and a layers panel to modify their ordering (A). This interface encourages spatial, imperative manipulation of objects on the canvas, which works well for these other design platforms. However, FIGMA's system is strongly declarative—it is primarily governed by complex constraints the user can impose and modify. These constraints manifest in *auto layout* through spacing and element positioning rules; in *components* through their linked instances, properties, and variants; and in *prototyping* through smart animate's reliance on hierarchical consistency and predefined visual states. We identified the following three themes that shape user interaction in FIGMA.

³<https://www.adobe.com/products/photoshop.html>

⁴<https://www.adobe.com/products/illustrator.html>

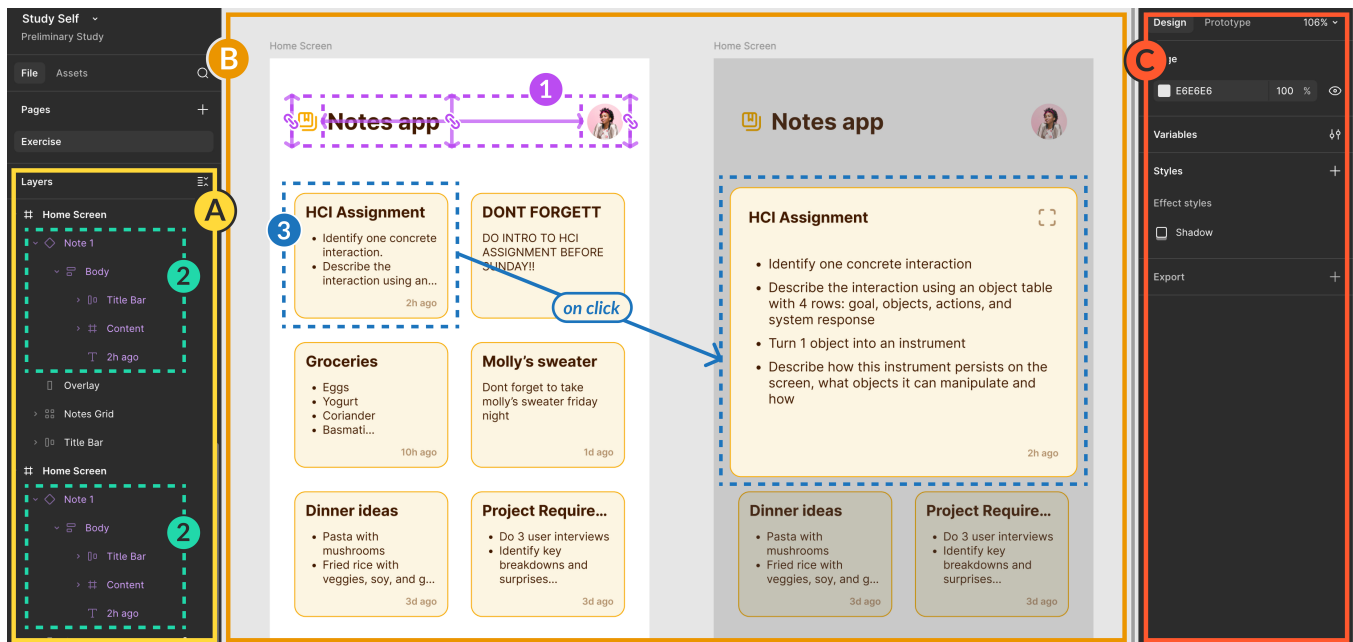


Figure 1: The FIGMA interface includes: (A) layers panel, (B) canvas, and (C) side panel. We identified three common breakdowns: (1) hidden constraints override user actions, (2) critical information is confined to the layers panel, and (3) prototyping is centered on designing transitions between frames rather than interactions. (This example illustrates the study task.)

4.1 FIGMA prioritizes user- and system-defined constraints over direct user actions

FIGMA is structured around a network of constraints—defined both by the user and by the system—that compute layouts and behaviors to produce a consistent and *expected* output. However, participants rarely conceptualized their actions in terms of constraint specification, as they attempted to override or adjust behaviors directly through manipulation on the canvas—which the system is not designed to support. In fact, six out of 13 participants struggled with the elements’ position anchors when resizing parent elements, as their children either “deform[ed]” (P4), or remained unchanged. The cause of these behaviors was the position anchor constraints applied to the children, which were either bound to an edge (the top-left corner for P1, P7, and P8) or set to scale on only one axis (P4). These constraints (see figure 1-1) were only visible in the side panel when selecting the children.

Similarly, seven participants faced problems with auto layout constraints and three struggled with its grid option, specifically when arranging notes in the home screen. All transformed a frame containing a set of notes into a single-column grid and resized the frame to allow space for a second column. When they attempted to move a note into the second column’s space, it slid back into the first column. Resizing the frame only changed its dimensions and did not update the constraints defining the grid, which remained a single-column layout. The grid constraints could only be modified through the side panel, which no participant attempted. Instead, they were confused and either gave up or resorted to workarounds, such as using a horizontal layout with wrapping (P7). Constraints

were often hidden from users, leading to confusion about interaction outcomes. In addition, users sometimes assumed that direct manipulation of objects would alter these constraints, which instead remained fixed and had to be explicitly changed through the side panel.

4.2 FIGMA treats the layers panel as the source of truth, not the canvas

Similar to many creativity-support tools that support spatial manipulation, FIGMA includes both a primary canvas and a layers panel on the side. This layout foregrounds the canvas as the main interaction space, where participants accordingly spent most of their time. This works well for tools such as ADOBE PHOTOSHOP and ILLUSTRATOR because it allows symmetric presentation of relevant information. However, FIGMA also exposes and manages crucial information in the layers panel. Five participants struggled with the ordering and nesting of objects, since they could not tell which of some non-overlapping siblings is above the others, or whether an object is inside a group. In contrast to figure 1-2, which shows two frames with matching layers, P6 and E9 created inconsistent layers, which in turn caused problems with smart animate and was “very annoying” (E9). P1, on the other hand, created a series of buttons in a frame by layering icons over circles, without grouping each set. Once she applied auto layout on the frame to space them out, it separated the icons from the circles.

Beyond these hierarchical relationships, the layers also hold relevant information for component variants, which are defined through structured properties: each variant’s name is a unique combination of property–value pairs as a comma-separated list

of *property=value*. While this mechanism simplifies the creation, extraction, and grouping of variants in large design systems, five participants encountered issues when renaming variants directly in the layers panel. They renamed variants using only the value, e.g., “image” or “text,” breaking the underlying property structure and triggering error messages they described as “confusing” (P1, P8). Only one participant resolved the issue without the researcher’s help. These observations highlighted a fundamental asymmetry in FIGMA between visual real-estate and structural importance, since the layers panel—tucked to the side and often overlooked by users—exposed critical information for smart animation, grouping, and component variants.

4.3 FIGMA frames prototyping as designing action-triggered animations, not designing interactions

Prototyping in FIGMA is primarily conceived as navigation across screens, triggered by user action. To create more complex effects, users can rely on smart animate, which emulates animation techniques commonly found in motion graphics software, such as ADOBE AFTER EFFECTS⁵. Smart animate lets users define two distinct visual states in separate frames and create a transition between them, allowing elements to morph from one state to another (see figure 1-3). While P6 found this “clear and easy,” three participants experienced a conceptual mismatch. They felt “as if [they’re] really animating instead of building a system” (P7), since they were not directly defining interaction logic, such as a click to move an element by a few pixels. P2 assumed that her prototype would playback automatically, like a timeline-based animation, and did not realize that she needed to actively interact with it. FIGMA’s prototyping feature is thus oriented more towards animating visual effects than designing complex interactions.

Interestingly, only one of the four experts was able to prototype a drag-and-drop, and even said he “didn’t know that [FIGMA] had this feature” (E13). He simulated the interaction by smart animating a transition between two frames using an *on drag* trigger on a specific element, with the latter appearing in a different position in the destination frame. However, E13 also noted that “it feels a bit weird that [the user] can only drag it to the desired destination,” since the element could not move freely according to the cursor’s position. E12 also attempted to prototype a drag-and-drop, but approached the task differently. His instinct was to extract the cursor position into a variable and dynamically assign it to the element’s position, treating interaction as a continuous mapping rather than a transition between states. Because this functionality was not available in the platform, E12 was unable to achieve the intended interaction.

5 Discussion

FIGMA is a powerful tool for designers to create responsive layouts and complex design systems. However, it sits between being a proper visual programming system and a traditional design tool. It presents constraints, parametrization, and other concepts related to programming environments, but does not fully commit and remains

tied to the familiar design tool layout. This positioning reflects a deliberate design trade-off: prioritizing familiarity—by abstracting prototyping as navigation across frames, for example—over explicitly communicating the underlying computational structure, which causes problems for users and sometimes limits their power of expression.

We can view the structure of a FIGMA project as a complex substrate [13] that contains all the elements in the canvas, maintains their states and hierarchical relationships, and imposes user- and system-defined constraints that dictate how these elements behave and can be manipulated. Our analysis of FIGMA using a generative theory lens [1] identified the following four implications for designing constraint-reliant tools.

Display contextual constraints. When the user is interacting with an object, ensure that all the constraints that affect this interaction are visible in the current view of the system. For example, when a user resizes a frame in FIGMA—a breakdown discussed in section 4.1—the system could show in the canvas the anchors of all the children (see figure 1-1). Then, if a child does not resize with the frame, the user will see that its anchor is set to the top-left of the parent and make the desired changes accordingly.

Reify [2] constraints into persistent, manipulable objects. Turn constraints into objects that remain on screen, display the information they hold, and allow the user to modify their values through direct manipulation. Continuing the previous example, the visible anchors could remain present in the canvas, but only visible when the user is interacting with or hovering over an object within its context. When the user drags this anchor, they can reposition it, updating the anchor position of the element it constrains accordingly. Applying this concept to FIGMA’s grid auto layout would tackle the second breakdown discussed in section 4.1.

Match information density across views. Display information equally in all the relevant views of the system, adapting the information’s representation for each view. The user will thus be able to properly situate themselves in the interface. For example, FIGMA could symmetrically display the type of objects in the layers, side panel, and canvas. It could also, when coupled with *displaying contextual constraints*, communicate matching layers when prototyping a smart animation in both the layers panel and the canvas—addressing the animation breakdown discussed in section 4.2.

Make prototyping object-centered. Instead of pushing users to create multiple frames of the same screen and transition between them to prototype interactions, treat each object as a state machine that, when acted on, goes to a new state and locally changes some aspects in the current frame. For example, to prototype hiding an object when clicking on it, the user could select this object in the frame and add to it a new *on click* action. The latter will create a new state for this object, allowing the user to define that, in this clicked state, the object should have an opacity of 0. This structure may better align with the mental models expressed in section 4.3.

This is a preliminary study with the aim of exploring observations of interaction breakdowns. It is limited by the inclusion of only four expert users. Because participants received a short tutorial immediately before the task, the observations may reflect short-term learning or priming effects. In addition, although we

⁵<https://www.adobe.com/products/aftereffects.html>

standardized procedures, remote and in-person contexts may have influenced persistence and help-seeking behavior.

6 Conclusion

We are interested in how UX designers engage with programming-like concepts. Our study of 13 designers examines how they prototype interactive applications in FIGMA, a widely used high-fidelity, constraint-based prototyping tool. We identified recurring breakdowns that stem from the inherent mismatch between an interface that encourages imperative manipulation of objects on the canvas and the system's underlying declarative structure, including: constraints often override users' actions; the layers panel often obscures critical information; and the prototyping model conflicts with the designer's mental model of the interaction. By characterizing FIGMA as a complex interaction substrate, we identified three implications for designing constraint-reliant interactive systems: constraints should be reified and displayed explicitly when affecting the interaction context; different system views should expose the same core information consistently; and prototyping should center on objects rather than on transitions among frames. We argue that constraint-based visual programming tools should thus explicitly communicate the underlying computational structure.

Acknowledgments

This work was supported by a French government grant managed by the Agence Nationale de la Recherche as part of the France 2030 program, reference ANR-22-EXEN-0003 (PEPR eNSEMBLE / PILOT).

References

- [1] Michel Beaudouin-Lafon, Susanne Bødker, and Wendy E. Mackay. 2021. Generative Theories of Interaction. *ACM Trans. Comput.-Hum. Interact.* 28, 6 (Nov. 2021), 45:1–45:54. doi:10.1145/3468505
- [2] Michel Beaudouin-Lafon and Wendy E. Mackay. 2000. Reification, polymorphism and reuse: three principles for designing visual interfaces. In *Proceedings of the working conference on Advanced visual interfaces (AVI '00)*. Association for Computing Machinery, New York, NY, USA, 102–109. doi:10.1145/345513.345267
- [3] Alan Borning. 1981. The Programming Language Aspects of ThingLab, a Constraint-Oriented Simulation Laboratory. *ACM Trans. Program. Lang. Syst.* 3, 4 (Oct. 1981), 353–387. doi:10.1145/357146.357147
- [4] Virginia Braun and Victoria Clarke. 2019. Reflecting on reflexive thematic analysis. *Qualitative Research in Sport, Exercise and Health* 11, 4 (Aug. 2019), 589–597. doi:10.1080/2159676X.2019.1628806 _eprint: https://doi.org/10.1080/2159676X.2019.1628806
- [5] Stéphane Chatty, Stéphane Sire, Jean-Luc Vinot, Patrick Lecoanet, Alexandre Lemort, and Christophe Mertz. 2004. Revisiting visual interface programming: creating GUI tools for designers and programmers. In *Proceedings of the 17th annual ACM symposium on User interface software and technology (UIST '04)*. Association for Computing Machinery, New York, NY, USA, 267–276. doi:10.1145/1029632.1029678
- [6] Jonathan Chen and Dongwook Yoon. 2024. Exploring the Diminishing Allure of Paper and Low-Fidelity Prototyping Among Designers in the Software Industry: Impacts of Hybrid Work, Digital Tools, and Corporate Culture. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3613904.3642774
- [7] Won Choi and Marta Larusdottir. 2023. Approaches and Challenges of Inclusive UX Practices in the Software Industry. In *Proceedings of the 36th International BCS Human-Computer Interaction Conference*. BCS Learning & Development Ltd, University of York, UK, 150–154. doi:10.14236/ewic/BCSHCI2023.18
- [8] Marianela Ciolfi Felice, Nolwenn Maudet, Wendy E. Mackay, and Michel Beaudouin-Lafon. 2016. Beyond Snapping: Persistent, Tweakable Alignment and Distribution with StickyLines. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (UIST '16)*. Association for Computing Machinery, New York, NY, USA, 133–144. doi:10.1145/2984511.2984577
- [9] K. J. Kevin Feng, Tony W Li, and Amy X. Zhang. 2023. Understanding Collaborative Practices and Tools of Professional UX Practitioners in Software Organizations. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–20. doi:10.1145/3544548.3581273
- [10] Tim Gorichanaz. 2025. Identifying the Barriers to Human-Centered Design in the Workplace: Perspectives from UX Professionals. *Proc. ACM Hum.-Comput. Interact.* 9, 2 (May 2025), CSCW110:1–CSCW110:25. doi:10.1145/3711008
- [11] Emily Kuang, Xiaofu Jin, and Mingming Fan. 2022. "Merging Results Is No Easy Task": An International Survey Study of Collaborative Data Analysis Practices Among UX Practitioners. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3491102.3517647
- [12] James Lin, Tiffany Kneare, and Kristopher Giesing. 2023. Relay: A collaborative UI model for design handoff. In *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23 Adjunct)*. Association for Computing Machinery, New York, NY, USA, 1–3. doi:10.1145/3586182.3616624
- [13] Wendy E. Mackay and Michel Beaudouin-Lafon. 2025. Interaction Substrates: Combining Power and Simplicity in Interactive Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3706598.3714006
- [14] Nolwenn Maudet, Germán Leiva, Michel Beaudouin-Lafon, and Wendy Mackay. 2017. Design Breakdowns: Designer-Developer Gaps in Representing and Interpreting Interactive Systems. In *Proceedings of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing (CSCW '17)*. Association for Computing Machinery, New York, NY, USA, 630–641. doi:10.1145/2998181.2998190
- [15] Brad A. Myers. 1990. Creating user interfaces using programming by example, visual programming, and constraints. *ACM Trans. Program. Lang. Syst.* 12, 2 (April 1990), 143–177. doi:10.1145/78942.78943
- [16] Brad A. Myers. 1990. Taxonomies of visual programming and program visualization. *Journal of Visual Languages & Computing* 1, 1 (1990), 97–123. doi:10.1016/S1045-926X(05)80036-9
- [17] Francesca Rossi, Peter van Beek, and Toby Walsh. 2008. Constraint Programming. In *Handbook of knowledge representation* (1st ed ed.). Elsevier, Amsterdam Boston, 181–212.
- [18] Prakash Shukla, Suchismita Naik, Ike Obi, Phuong Bui, and Paul Parsons. 2024. Communication Challenges Reported by UX Designers on Social Media: An Analysis of Subreddit Discussions. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3613905.3650881
- [19] Steven L. Tanimoto. 1990. VIVA: A visual language for image processing. *Journal of Visual Languages & Computing* 1, 2 (June 1990), 127–139. doi:10.1016/S1045-926X(05)80012-6
- [20] Shutong Zhang, Tianyu Zhang, Jinghui Cheng, and Shurui Zhou. 2025. Who is to Blame: A Comprehensive Review of Challenges and Opportunities in Designer-Developer Collaboration. *Proc. ACM Hum.-Comput. Interact.* 9, 2 (May 2025), CSCW207:1–CSCW207:32. doi:10.1145/3711105